

Mauro Ruiz

PROJETO RESTAURANTE - ENTREGA SEQUENCIAL 2

(novo a partir do item3)

Concepção do Sistema

1) Premissas do Negócio e Requisitos

- **Tipo de restaurante:** Restaurante familiar de bairro próximo à Fatec, focado em atendimento rápido de almoço para professores e alunos com pouco tempo.
- **Produtos ofertados:** Prato Feito (PF), sanduíche frio (cada um de um único tipo) e 3 categorias de bebidas (água, suco e refrigerante de apenas 1 marca cada, todas padronizadas em volume único de 500ml).
- **Preços por Categoria:** Preço fixado pela categoria da bebida, independentemente da marca escolhida.

2) Arquitetura Geral do Sistema

```
[ SISTEMA DO RESTAURANTE ]
|
+--- Bloco 1: O CARDÁPIO E ESTOQUE (Herança, Encapsulamento e FIFO)
|   |--- Produto (Classe Base / Mãe)
|   |--- Comida (Classe Filha)
|   |--- Bebida (Classe Filha)
|   |--- LoteEstoque (Registra quantidade, dataCompra e dataVencimento)
|   +--- Estoque (Baixa PEPS/FIFO: consome lotes mais velhos primeiro)
|
+--- Bloco 2: O FLUXO DE ATENDIMENTO (Comanda e Fila Personalizada)
|   |--- Comanda (Dados do pedido, cliente e status)
|   +--- FilaAtendimento (Estrutura FIFO própria para os PFs)
|
+--- Bloco 3: O CAIXA E HISTÓRICO (Pagamentos e Vendas)
|   |--- Pagamento (Registra valor, data e forma: Pix, Cartão, Dinheiro)
|   +--- Caixa (Processa pagamentos e armazena o histórico)
|
+--- Bloco 4: REPOSIÇÃO, DADOS E RELATÓRIOS (Gestão e Persistência)
|   |--- GerenciadorEstoque (Verifica e repõe mercadoria)
|   |--- GerenciadorDados (Gera dados com Faker e persiste com Pickle)
|   +--- GerenciadorRelatorios (Gera relatórios de vendas e consumo)
```

Minha pergunta:

Como explicitado no quadro acima, temos Produto como uma classe mãe e Comida e Bebida, respectivamente como classes filhas. Pelo meu entendimento, baseando-me nas premissas iniciais, os objetos nesse bloco para a classe filha Comida são: Prato feito, sanduíche; e para a classe Bebida são: Água, suco e refrigerante. Estou certo ou tem algum reparo até aqui? Caso eu esteja no caminho certo, me mostre como ficaria o código para o primeiro bloco apenas, pois estou tendo algumas dificuldades iniciais.

Resposta da IA:

O seu entendimento sobre o domínio do negócio está correto em termos funcionais, mas do ponto de vista da Programação Orientada a Objetos (POO), **vale um pequeno ajuste de modelagem** antes de ir para o passo a passo para implementação do Bloco 1.

Classes x Instâncias: Comida e Bebida são as classes filhas. "Prato Feito", "Sanduíche Frio", "Água", "Suco" e "Refrigerante" não precisam ser subclasses separadas, mas sim instâncias (objetos) dessas classes.

Controle de Estoque (PEPS/FIFO): O estoque armazena lotes para controlar validade e quantidade.

3) Passo a passo explicativo para a implementação do Bloco 1

Passo a passo detalhado em linguagem natural de como construir para implementar o **Bloco 1**:

Passo 1: Importar a ferramenta de datas

- **O que fazer:** Logo no início do arquivo, importe apenas a funcionalidade date da biblioteca datetime.
- **Por quê:** Ela vai permitir criar e manipular datas de compra e vencimento sem precisar incluir horas e minutos.

Passo 2: Criar a classe mãe Produto

- **O que fazer:** Defina a classe base chamada Produto. No método de inicialização (`__init__`), declare três parâmetros: o código do produto (`id_produto`), o nome (`nome`) e o preço (`preco`).
- **Por quê:** Todo item vendável no restaurante compartilha essas três características fundamentais.

Passo 3: Criar as classes filhas Comida e Bebida (Herança)

- **O que fazer:**
 1. Crie a classe Comida herdando de Produto. Na inicialização, receba o `peso_g` e use a instrução `super().__init__()` para repassar o `id`, `nome` e `preço` para a classe mãe.

2. Crie a classe Bebida herdando de Produto. Na inicialização, receba a categoria (ex: "Água", "Suco", "Refrigerante"). Ao chamar o `super().__init__()`, passe a própria categoria como o nome do produto. Defina o atributo `volume_ml` diretamente como 500 por padrão dentro da classe.
- **Por quê:** Isso atende à premissa do negócio de que bebidas têm volume padronizado em 500ml e preços fixados por categoria.

Passo 4: Criar a classe LoteEstoque

- **O que fazer:** Crie uma classe que receba e guarde quatro informações: a referência do produto (o objeto criado), a quantidade recebida, a `data_compra` e a `data_vencimento` (ambas utilizando objetos `date`).
- **Por quê:** Para o controle de estoque funcionar, você não pode guardar apenas a quantidade total; precisa guardar as caixas / lotes individuais com suas respectivas datas.

Passo 5: Criar a classe Estoque e a lista de controle

- **O que fazer:** Crie a classe Estoque. No método `__init__`, crie uma lista vazia chamada `self.lotes`.
- **Por quê:** Essa lista servirá como o repositório central onde todos os lotes que chegarem serão armazenados.

Passo 6: Implementar a inserção de lotes

- **O que fazer:** Crie o método `adicionar_lote(self, lote)`. Dentro dele, pegue o lote recebido e faça um `.append()` na lista `self.lotes`.
- **Por quê:** Conforme novas remessas chegam, elas entram no final da lista, mantendo a ordem de chegada.

Passo 7: Implementar a lógica de baixa PEPS/FIFO

- **O que fazer:** Crie o método `dar_baixa_fifo(self, id_produto, quantidade_desejada)`:
 1. Crie uma variável temporária `qtd_restante` para acompanhar quanto ainda falta dar baixa.
 2. Faça um laço de repetição `for` para percorrer a lista de lotes do início ao fim (ou seja, dos mais antigos para os mais novos).
 3. Dentro do laço, verifique se o lote pertence ao produto correto (`id_produto`) e se ele ainda tem itens disponíveis (`quantidade > 0`).
 4. Se o lote atual tiver **menos ou igual** à quantidade necessária, zere o lote e subtraia o valor consumido de `qtd_restante`.
 5. Se o lote atual tiver **mais** itens do que o necessário, subtraia a necessidade do lote e zere `qtd_restante`.
 6. Quando `qtd_restante` chegar a zero, interrompa o laço com `break`.
 7. Se o laço terminar e `qtd_restante` ainda for maior que zero, exiba uma mensagem alertando que não havia estoque suficiente.

- **Por quê:** Essa lógica garante que os lotes comprados primeiro (que estão no início da lista) sejam consumidos até zerar antes de passar para o lote seguinte.